

Nexus Logistics — Hackathon Project Description

Nexus Logistics — Master Project Description

Avishkara'26 · JP-019 · Team: Code with Errors

Purpose: Source material for hackathon PPT (repository-verified, v2 branch).

Inspection date: 2026-09-09

Rule: Claims below are traced to the current codebase. Unverified items are marked **NOT VERIFIED**.

REPOSITORY VERIFICATION STATUS

Area	Status
Product identity / team	VERIFIED (presentation + landing copy)
Primary UX = Presentation + Optimizer Lab	VERIFIED
V1 Console as /optimizer	VERIFIED
Synthetic day → SQLite lab	VERIFIED
Lab optimize via /api/solve + /api/compare	VERIFIED
/api/lab/run exists	VERIFIED
/api/lab/run used by Console UI	NOT USED (helper exists; Console does not call it)
CVRPTW + OR-Tools 9.11.4210	VERIFIED
PATH_CHEAPEST_ARC / GUIDED_LOCAL_SEARCH	VERIFIED (numeric enums 3 / 2 with comments)
GradientBoostingClassifier late-risk	VERIFIED

Area	Status
OSRM + Haversine	VERIFIED
Synthetic training data + not_production_validated	VERIFIED
Console vehicle-breakdown / reoptimize UI	NOT IMPLEMENTED in Lab Console
Ops Exceptions breakdown (separate auth app)	VERIFIED (exists, not primary Lab)
Presentation session metrics auto-written from Lab	NOT VERIFIED / UNUSED (writeLabSession never called)
Live traffic / GPS tracking	NOT IMPLEMENTED
External LLM (OpenAI/Gemini/ etc.)	NOT PRESENT
Production deployment metrics	NOT VERIFIED

1. EXECUTIVE PROJECT DESCRIPTION (~200 words)

Nexus Logistics is a last-mile delivery decision-support system for urban fleets (demo geography: **Bengaluru**). It helps a dispatcher-style operator turn a day's delivery orders and van fleet into a **feasible route plan** and a **late-delivery risk view**.

Problem: Last-mile planning is not “find the shortest path.” Real days combine many orders, limited vans, **hard capacity**, **hard customer time windows**, priorities (critical vs normal), and the risk that some stops arrive late—even when a short tour exists.

Who uses it (hackathon story): Operators / planners demonstrating constraint-aware routing and risk intelligence—not a full ERP (admin/dispatcher/driver dashboards exist in-repo but are **outside** the primary Presentation + Lab journey).

Inputs: Orders (location, demand, time window, priority), vehicles (capacity, shift, depot), optional holds, travel matrices (OSRM or Haversine).

Processing: A CVRPTW solver (**Google OR-Tools**) builds routes under hard capacity and windows. Soft drop costs allow deferring stops that cannot fit. Separately, a **GradientBoostingClassifier** scores late-delivery risk (pre-route triage + post-route overlay). Risk may slightly influence which non-critical stops are preferred when dropping—but **never relaxes hard constraints**.

Outputs: Van assignments and stop sequences, map polylines, scoreboard metrics (distance, late count, breaches, deferred, etc.), risk percentages and reasons, optional baseline-vs-optimized comparison.

Why useful: It demonstrates measurable, constraint-true planning: fewer invalid “short” tours, clearer deferred work, and advisory late-risk—so planners see *what can be done* and *what may go wrong*.

2. PROBLEM STATEMENT

Last-mile delivery in a city like Bengaluru requires simultaneous decisions:

- **Where** each stop is
- **When** it must arrive (time window)
- **How much** demand it adds (capacity)
- **Which van** serves it
- **In what sequence** stops are visited
- Whether a stop is **critical**
- Whether the plan will still be **late** under operational features

A tour that minimizes distance alone can still be **invalid**: overloaded vans, missed windows, or leftover stops when the fleet cannot cover everything.

Concrete product example (Day B / overloaded pack): When the fleet is overconstrained, the optimizer returns a **partial plan**—criticals are protected where possible; remaining stops are **deferred** with reasons such as capacity/window pressure (UI humanizes codes like `deferred_capacity_or_window_high_late_risk`). Operators can **hold** stops in SQLite and re-solve.

Vehicle disruption in the Lab Console: **NOT IMPLEMENTED** as a one-click “simulate breakdown → reoptimize” flow. A related **ops Exceptions** path exists under auth (mark breakdown / reoptimize-live)—that is **not** the primary hackathon Lab Console.

3. PROPOSED SOLUTION

INPUT → Orders, vehicles, depot, holds, weather context (optional refresh).

PROCESSING → Load scenario (static Day A/B CSVs seeded into SQLite, or synthetic lab pack). Build travel time/distance matrices.

OPTIMIZATION → Baseline heuristic (critical-first greedy + nearest-neighbor) **or** OR-Tools CVRPTW (PATH_CHEAPEST_ARC + GUIDED_LOCAL_SEARCH). Soft disjunctions defer infeasible stops.

ML RISK → Pre-route triage scores; post-route overlay attaches `p_late`, reasons, triage (monitor / intervene). Soft triage may bump normal drop cost.

OUTPUT → Routes on Leaflet map, scoreboard, deferred list, risk inspector, compare table (naive vs smart), exports (manifest/geojson), win sheet API.

4. COMPLETE SYSTEM WORKFLOW (as implemented for Lab)

Primary hackathon Lab path (Console):

1. Optional: **Synthetic day** → POST /api/lab/synthetic → persists scenario id **lab** in SQLite (new generation each click; generation_id / seed).
2. Load UI scenario → GET /api/scenarios/{id} (a | b | lab).
3. Distance matrices inside solve (OSRM if available, else Haversine; JP019_TRAVEL=haversine forces offline).
4. **Naive plan** → POST /api/solve {scenario_id, mode:"baseline"}.
5. **Smart optimize** → POST /api/solve {scenario_id, mode:"optimize"} (OR-Tools + risk attach).
6. **Compare both** → POST /api/compare?scenario_id=... (baseline + optimize + improvement rows).
7. Frontend: CityMap polylines, VehicleRail, DeferredList, RiskInspector, Scoreboard, OrderBoard.
8. Holds → POST /api/holds / clear holds → re-solve.

Also exists but NOT the Console primary path:

- POST /api/lab/run — in-memory payload baseline+optimize+comparison; used in backend tests; runLabOptimize helper unused by Console.

Breakdown → reoptimize in Console: NOT PRESENT.

Ops path (auth): Exceptions page + /api/ops/optimize-live with exclusions — **secondary**, not Lab nav.

5. EXACT TECHNOLOGY STACK

FRONTEND

- Framework: **React 19.1.0**
- Language: **TypeScript 5.8.3**
- Build: **Vite 6.3.5**
- Routing: **react-router-dom ^7.18.3**
- Styling: **Tailwind CSS 3.4.17**
- Maps: **Leaflet ^1.9.4**
- Icons: **lucide-react 0.511.0**
- Auth client: **firebase ^12.18.0**

BACKEND

- Framework: **FastAPI 0.115.8**
- Language: **Python**
- Server: **Uvicorn 0.34.0**

- Validation: **Pydantic 2.10.6**
- Optimization: **ortools 9.11.4210**
- ML: **scikit-learn 1.6.1, joblib 1.4.2, pandas/numpy**
- Cache: **redis** optional (in-memory fallback)
- Admin: **firebase-admin 6.5.0**

DATABASE / STORAGE

- **SQLite** (`nexus.db`): scenarios a/b/lab, orders, vehicles, holds, solve_runs history
- **Firestore**: operational entities / users / realtime ops (secondary to Lab story)
- **CSV**: `backend/data/history.csv` (~420 labeled rows) for ML training
- **Artifacts**: `backend/data/models/late_risk.joblib` + `late_risk_meta.json`

AUTHENTICATION

- Firebase Auth ID tokens for ops APIs
- **Lab routes auth-optional** (no login required for `/optimizer`, `/api/lab/*`, `/api/solve`, `/api/compare`)

ROUTING / DISTANCE

- OSRM public table API when available
- Haversine fallback (~25 km/h average for minutes)

ML / OPTIMIZATION

- `GradientBoostingClassifier` (+ `LogisticRegression` fit fallback)
 - Google OR-Tools `RoutingModel CVRPTW`
-

6. OPTIMIZATION ALGORITHM

Problem name: CVRPTW — Capacitated Vehicle Routing Problem with Time Windows.

Hard constraints (verified):

1. Vehicle **capacity** dimension
2. Order **time windows** [`tw_start`, `tw_end`]
3. Vehicle **shift** windows on start/end

Decision variables (solver): which vehicle visits which order nodes; visit order; cumulative time/load.

Objective: Minimize arc routing cost (travel+service transit callback), with **soft disjunctions** (drop penalties) so some orders may be deferred rather than forcing an infeasible hard assignment.

Initial solution: first_solution_strategy = 3 → **PATH_CHEAPEST_ARC** (commented in code for ortools 9.11).

Improvement: local_search_metaheuristic = 2 →

GUIDED_LOCAL_SEARCH.

Time limit: default **8s** (Lab run path uses **10s**).

Vehicles: capacity, depot lat/lon, shift_start/end, id, driver/plate metadata.

Deliveries: order_id, lat/lon, demand, tw_start/end, service_min, priority.

Deferred: orders not in any route after solve; constraint log reasons; partial flag when unassigned remain.

Soft late-risk influence: For non-critical orders, drop penalty += up to $2000 \times p_late$ from pre-route triage—prefer keeping riskier normals when capacity allows; **does not loosen capacity/windows**.

7. WHY THIS ALGORITHM?

A. 20-second explanation

“We don’t only shorten distance. Vans have **kg limits** and customers have **arrival windows**. CVRPTW is the standard model for that. OR-Tools searches for a **feasible** plan and can **defer** stops that won’t fit—without cheating the rules.”

B. Technical explanation

CVRP alone ignores time windows; pure TSP ignores capacity. CVRPTW jointly encodes capacity dimensions and time cumulatives with hard ranges. Soft disjunctions implement operational reality: when overconstrained, produce a **partial feasible** plan rather than violating hard constraints. **PATH_CHEAPEST_ARC** seeds a constructive solution; **GUIDED_LOCAL_SEARCH** improves under the metaheuristic within a wall-clock limit. ML only adjusts soft drop costs for normals—hard constraints remain hard.

8. MACHINE LEARNING

Item	Verified value
Model	GradientBoostingClassifier (n_estimators=80, max_depth=3, lr=0.08, random_state=19)
Library	scikit-learn
Fit fallback	LogisticRegression if GB fit throws
Purpose	Late-delivery risk probability p_late
Architecture	Pre-route triage + post-route overlay
Training data	Synthetic CSV history.csv

Item	Verified value
Target	was_late
Artifact	late_risk.joblib + meta JSON
Validation flag	not_production_validated: true
Holdout (meta)	train 315 / test 105; accuracy ~0.71, precision ~0.76, recall ~0.85, F1 0.80, ROC-AUC ~0.77 (synthetic holdout— not production)

9. ML FEATURES

Post-route overlay (FEATURES) — LIVE after a plan exists

Feature	Meaning	Why it matters
depot_km	Distance from depot	Longer legs → more lateness risk
window_width	Tw end – start	Narrower windows harder to hit
load_pct	Planned load vs capacity	Near-full vans correlate with pressure
seq_index	Stop sequence index	Later stops accumulate delay risk
demand	Order demand	Heavier stops / denser packing
zone_late_rate	Zone historical late rate (from train)	Area-level lateness pattern

Pre-route triage (PRE_ROUTE_FEATURES) — LIVE before routing

depot_km, window_width, demand, zone_late_rate (**no** load_pct / seq_index).

Excluded

eta_slack — depends on planned ETA; excluded from models (documented in meta).

10. ML TRAINING DATA

- Source: backend/data/history.csv
- Nature: **Synthetic demonstration data** (explicit disclosure in code/meta)
- ~420 labeled rows
- Split: 75/25 stratified on was_late, random_state=19
- Labels: was_late (+ delay_min in CSV)

- Limitation: **not production validated**; do not claim real-ops accuracy in PPT
-

11. DISTANCE / ROUTING

- **OSRM** table API for road durations/distances when reachable
 - **Haversine** fallback (and forced via JP019_TRAVEL=haversine|offline|demo)
 - Why needed: CVRPTW arc costs and ETAs
 - **Live traffic**: NOT IMPLEMENTED
 - **GPS tracking**: NOT IMPLEMENTED
 - **Congestion feeds**: NOT IMPLEMENTED
-

12. SYNTHETIC DATA

generate_lab_scenario / POST /api/lab/synthetic:

- Bengaluru depot ~12.9716, 77.5946
- Zones: Indiranagar, Koramangala, Jayanagar, HSR, Whitefield, MG Road
- **4–6 vans, 14–25 orders**; ~22% critical
- Demand soft-capped ~65% of fleet capacity
- Each call: new seed / generation_id; overwrites SQLite scenario **lab**
- **Statement verified**: Data is synthetic; pipeline (matrices → OR-Tools → risk → map) is the real application stack

Static packs **Day A** / **Day B** also exist (CSV-seeded SQLite) for controlled demos.

13. OPTIMIZER LAB (CURRENT)

- Route: **/optimizer** (alias **/lab** → redirect)
- UI: **V1 Console** (Console.tsx) + thin return-to-presentation chrome
- Purpose: Prove naive vs smart planning on Bengaluru scenarios

Working flow (verified):

1. Pick **Day A**, **Day B**, or click **Synthetic day** (new data each click)
2. **Naive plan** / **Smart optimize** / **Compare both**
3. View **scoreboard**, **map**, **vans**, **deferred**, **risk inspector**, order board
4. Optional **holds**, weather refresh, play-day animation, exports, win sheet

NOT in Console Lab: one-button Simulate Breakdown → Reoptimize (ops Exceptions only).

14. BASELINE VS NEXUS

Baseline (run_baseline): Sort critical-first by `tw_end`; greedy pack by remaining capacity; nearest-neighbor sequencing per van. May **violate capacity** (intentional weak control).

Nexus optimize: OR-Tools CVRPTW with hard capacity/windows + soft drops + risk attach.

Compared metrics (compare.py): `distance_km`, `time_min`, `late_count`, `avg_lateness_min`, `tw_violations`, `capacity_breaches`, `unassigned_count`, `criticals_served`, `vehicles_used` (+ deltas include `hard_breaches`).

Do not hardcode PPT numbers—show live Lab run values only.

15. VEHICLE BREAKDOWN / REOPTIMIZATION

Lab Console

NOT IMPLEMENTED as productized breakdown demo.

Related capability (ops, auth)

Exceptions / optimize-live can exclude vehicles and re-run—**NOT VERIFIED** as the slide-7 Lab story unless you demo the ops path separately. Prefer honesty: Lab shows **overconstraint** → **deferred** → **hold** → **re-solve**; full breakdown UI is ops-side.

16. FRONTEND ARCHITECTURE

- `PresentationPage` — 9 interactive slides
 - `OptimizerLabPage` → Console
 - `CityMap` (Leaflet), `VehicleRail`, `Scoreboard`, `RiskInspector`, `DeferredList`, `ComparisonTable`, `OrderBoard`, exports
 - Talks to FastAPI via `api.ts` / `labApi.ts` + optional Firebase bearer
-

17. BACKEND ARCHITECTURE

Frontend (React)
↓ HTTP JSON
FastAPI (main.py + ops_api)
↓
SQLite scenarios / holds / history
↓
distance (OSRM|haversine) → baseline | optimizer (OR-Tools)
↓
risk model attach / triage
↓
SolutionOut JSON → map + metrics

18. DATABASE / STORAGE ARCHITECTURE

Store	Role
SQLite	Lab scenarios a/b/lab, holds, solve_runs
Firestore	Ops entities, users, realtime demos
CSV history	ML training labels
joblib meta	Persisted classifiers + holdout metrics

19. SECURITY / AUTH

Firebase Auth + RBAC for **ops** workspaces. **Hackathon Lab is intentionally auth-optional.** Keep PPT focus on optimization+ML, not IAM.

20. NOVELTY / DIFFERENTIATION (grounded)

1. **Hard CVRPTW** with explicit soft deferral (partial plans that don't cheat).
2. **Baseline vs OR-Tools** comparison on the same scenario (measured, not hardcoded).
3. **ML late-risk** as advisory overlay + soft triage—not as the router.
4. **Synthetic Bengaluru generator** feeding the **same** SQLite solve path as Day A/B.

21. WHAT ACTUALLY MAKES THIS “AI/ML”

Engine	Does	Does NOT
OR-Tools	Assigns vans/ sequences under hard capacity & windows	Predict customer lateness probability
GradientBoosting	Estimates p_late; triage labels; soft drop bias for normals	Override capacity/windows; invent routes alone

ML does not generate the route plan. Optimization does. ML informs risk and soft preferences.

No LLM.

22. JUDGE QUESTIONS (20)

1. **Which algorithm?** CVRPTW via Google OR-Tools.
2. **Why CVRPTW?** Capacity + time windows are first-class hard constraints.
3. **Why OR-Tools?** Mature VRP solver; PATH_CHEAPEST_ARC + GUIDED_LOCAL_SEARCH.
4. **What is Gradient Boosting?** Ensemble of shallow trees for classification of late vs not.
5. **Why GB?** Fits tabular operational features; sklearn implementation in-repo.
6. **Features?** depot_km, window_width, demand, zone_late_rate (+ load_pct, seq_index post-route).
7. **Training data?** Synthetic CSV (~420 rows), disclosed.
8. **Is data real?** No—synthetic demo history.
9. **Is this AI?** Optimization + supervised ML risk—not generative AI.
10. **Does ML generate routes?** No.
11. **No feasible full plan?** Soft drops → deferred/partial; hard constraints intact.
12. **Capacity?** Hard OR-Tools capacity dimension.
13. **Time windows?** Hard cumul ranges on Time dimension.
14. **Vehicle breaks down?** Lab Console: not one-click; ops Exceptions can exclude vans (**clarify**). Overconstraint demo via Day B + holds.
15. **Baseline?** Critical-first greedy + NN; may breach capacity.
16. **Distance?** OSRM else Haversine.
17. **OSRM down?** Automatic Haversine fallback / env force.
18. **Frontend?** **backend?** REST JSON to FastAPI.

19. **Scale?** Demo-sized scenarios (tens of stops); production scale **NOT VERIFIED**.
20. **Limitations?** Synthetic ML; not_production_validated; Lab $\neq$ full ops ERP; no live traffic/GPS.
-

23. 30-SECOND PITCH

“Last-mile routing isn’t shortest path—it’s capacity, time windows, and late risk. Nexus turns Bengaluru delivery scenarios into feasible van plans with Google OR-Tools CVRPTW, then scores late-delivery risk with Gradient Boosting. In our Optimizer Lab we generate fresh synthetic days, compare naive vs smart plans on a live map—rules stay hard, results stay measured.”

24. 60-SECOND PITCH

“Avishkara JP-019: last-mile delivery fails when plans ignore capacity and time windows. Nexus Logistics models the day as CVRPTW and solves it with Google OR-Tools—PATH_CHEAPEST_ARC then GUIDED_LOCAL_SEARCH—so vans never silently break hard rules; leftover stops are deferred. Separately, a GradientBoostingClassifier predicts late-delivery risk before and after routing, without overriding constraints. Our Optimizer Lab loads Day A, Day B, or a new synthetic Bengaluru pack each click, runs naive vs smart comparison, and shows routes, metrics, and risk on a Leaflet map. No LLM—mathematical optimization plus classical ML.”

25. 2-MINUTE TECHNICAL EXPLANATION

Input: orders (geo, demand, windows, priority) and vehicles (capacity, shifts) from SQLite scenario a/b/lab.

Travel: OSRM table or Haversine matrices.

Baseline: critical-first greedy pack + nearest neighbor (control).

Optimize: OR-Tools RoutingModel—capacity dimension, time dimension with hard windows, soft disjunctions with priority drop penalties; optional triage bump on normals; PATH_CHEAPEST_ARC + GUIDED_LOCAL_SEARCH within time limit.

ML: GB classifier on synthetic history; pre-route features for triage; post-route features including sequence/load; attach p_late and reasons to stops.

Output: JSON routes/metrics → React Lab map and scoreboard; compare API yields

deltas.

Auth-optional Lab; Firestore ops secondary. Honest limit: ML not production-validated; demo data synthetic; Lab breakdown button not present.

26. PPT CONTENT RECOMMENDATION (9 slides)

#	Title	Purpose	Key content	Visual	Interaction	Say	En
1	Nexus + team	Identity	JP-019, Code with Errors	Network graphic	Enter	Who we are	Be
2	Why hard	Problem	Capacity, windows, sequence	Decision board	Click constraints	Shortest ≠ feasible	Ha
3	CVRPTW	Formalize	Inputs/decisions/ constraints	Diagram	Highlight constraints	Name the problem	CV
4	Architecture	System	FastAPI, OR-Tools, GB, OSRM	Flow	Click components	How pieces connect	No
5	OR-Tools	Algorithm	PATH_CHEAPEST_ARC, GLS	Conceptual route	Watch conceptual	How search works	Ha
6	Late-risk ML	ML	Features, triage, disclosure	Risk bands	Tech details	What ML does/ doesn't	no
7	Live Lab	Proof	Open /optimizer	Lab CTA	Open Lab	Demo time	Re
8	When plans change	Disruption	Deferred/holds story (honest)	Steps	Replay	Re-plan without cheating	Pa
9	Impact + close	Close	Capabilities + live metrics if any	Checklist	—	Thank you	M

27. REAL DATA / REAL RESULTS (safe to show)

Metric	Source
orders / vehicles / critical / demand	Scenario payload / GET /api/scenarios/{id}

Metric	Source
distance_km, time_min, late_count, hard_breaches, capacity_breaches, tw_violations, unassigned_count, criticals_served, utilization	Solution metrics from solve/compare
p_late, reasons, triage	Stop risk after attach
travel_source	osrm / haversine / fallback
feasible / partial	Solution flags
Holdout ML metrics	late_risk_meta.json / /api/risk/metrics — label as synthetic holdout

Never invent km/late counts for PPT slides.

28. LIMITATIONS

DEMO

- Synthetic orders/history
- Public OSRM may be slow/unavailable → Haversine
- Solve time ~seconds–tens of seconds
- Presentation auto-metrics from Lab currently **not wired** (writeLabSession unused)
- No Lab breakdown button

PRODUCTION

- ML not_production_validated
- Scale beyond demo sizes NOT VERIFIED
- No live traffic/GPS
- Ops auth/Firestore complexity outside primary story

29. MASTER PROJECT DESCRIPTION (for PPT-generation AI)

Title: Nexus Logistics — Last-Mile Delivery Route Optimizer & Late-Delivery Risk Predictor (JP-019)

Problem: Urban last-mile planning must jointly respect vehicle capacity, delivery time windows, priorities, and late risk. Shortest-path routing alone produces invalid or brittle plans.

Solution: Nexus converts a day's orders and fleet into a CVRPTW instance, solves it with Google OR-Tools (PATH_CHEAPEST_ARC + GUIDED_LOCAL_SEARCH) under hard capacity and windows with soft deferral, and overlays GradientBoosting late-delivery risk (pre-route triage + post-route scores) without relaxing constraints.

How it works: Synthetic or packaged Bengaluru scenarios persist in SQLite; FastAPI serves solve/compare; travel via OSRM or Haversine; React Lab visualizes routes, metrics, deferred stops, and risk.

Stack: React/Vite/TS/Tailwind/Leaflet · FastAPI/Pydantic/Uvicorn · OR-Tools 9.11 · scikit-learn GB · SQLite + Firestore + CSV history · Firebase Auth for ops (Lab auth-optional).

Key features: Day A/B + synthetic generator; naive vs smart compare; map + risk inspector; holds; partial-plan honesty.

Innovation: Constraint-true optimization + advisory ML risk + measured baseline comparison on the same scenario.

Impact: Feasible plans, visible deferred work, late-risk awareness for dispatcher-style decisions.

Limitations: Synthetic data; ML not production-validated; no live traffic/GPS; Lab breakdown UI not implemented (ops-only).

Demo flow: Presentation → Open Lab → Synthetic day → Compare both → Inspect map/deferred/risk → return to presentation.

30. ONE-PAGE CHEAT SHEET

PROJECT: Nexus Logistics (Avishkara'26 · JP-019 · Code with Errors)

PROBLEM: Last-mile needs capacity + time windows + priorities—not just shortest distance.

ALGORITHM: CVRPTW · Google OR-Tools 9.11 · PATH_CHEAPEST_ARC + GUIDED_LOCAL_SEARCH

WHY: Hard capacity & windows; soft defer when overconstrained.

ML: GradientBoostingClassifier · late-delivery p_{late} · pre-route + post-route · synthetic history · **not_production_validated**

WHY ML: Predict what may go wrong; does **not** override hard constraints or invent routes alone.

TECH STACK: React/Vite/TS/Leaflet · FastAPI · OR-Tools · sklearn · SQLite/Firestore/CSV · OSRM+Haversine · no LLM

INPUT: Orders, vans, windows, demand, priority ($\pm$ holds)

OUTPUT: Feasible/partial routes, metrics, map, risk, baseline comparison

KEY FEATURE: Optimizer Lab—Day A/B + Synthetic day → Naive/Smart/
Compare on real APIs

BREAKDOWN: **Not in Lab Console;** Day B deferred + holds = re-plan story; ops
Exceptions separate

DATA: Synthetic Bengaluru scenarios + synthetic ML CSV (~420 rows)

LIMITATION: Synthetic ML; no GPS/traffic; don't invent metrics

30-SECOND ANSWER: “We solve CVRPTW with OR-Tools so capacity and
windows stay hard, then score late risk with Gradient Boosting—Lab shows
measured naive vs smart routes on a Bengaluru map. No LLM.”

End of repository-verified project description.